

PRD: AI Support Intelligence Agent

1) Product overview

The product is not a traditional support chatbot. It is a support intelligence layer that sits on top of customer conversations and turns them into operational decisions for support and engineering. The uploaded research context already points in the right direction: existing support bots mainly answer queries, reduce response times, and route tickets, but they do not reliably identify recurring issues, predict future incidents, or create actionable engineering signals.

In simple terms, the product listens to support conversations, extracts structure from them, detects patterns, and generates useful outputs such as issue clusters, sentiment, priority, summaries, and engineering tickets. The main value is not “chat”; the main value is faster discovery of product problems and better prioritization of fixes. That is why the product is stronger than a normal FAQ bot. The research context also recommends RAG, fallback flows, and human handoff as part of a serious support system.

2) Overall idea

The system should work like this:

Customer message → AI analyzes the conversation → AI classifies the issue → AI groups similar cases → AI estimates severity and sentiment → AI creates a clean engineering-ready report.

The product should serve three roles at once:

- Support assistant for the customer.
- Analysis assistant for the support team.
- Signal generator for engineering.

That makes the product more than a chatbot. It becomes a bridge between customers, support, and engineering.

3) Problem statement

Support teams usually handle tickets one by one. That solves the immediate customer issue, but it does not help the company understand the bigger picture. Repeated complaints, pattern shifts after deployment, urgency, customer frustration, and likely impact on revenue are often missed until later. The uploaded context already identified this gap and framed the opportunity around support intelligence rather than ordinary response automation.

4) Product goals

The product should:

- Reduce manual effort in understanding tickets.
- Detect repeated or related issues automatically.
- Summarize what is wrong in a way engineering can act on.
- Prioritize high-risk or frustrated customers.
- Create a simple, visible workflow from support to engineering.

5) Non-goals for the first version

Do not try to build everything at once.

- No full CRM replacement.
- No voice support.
- No enterprise-grade permissions system.
- No multi-channel omnichannel support in MVP.
- No complex agentic autonomy.
- No advanced revenue prediction in v1.

6) Bare basics required first

This is the minimum version that must exist before anything else.

1. Ticket intake
The user enters a support message.
2. Structured extraction
The AI converts the message into fields like:
 - category
 - summary
 - severity
 - sentiment
 - probable component
3. Storage
Save the message and the extracted result.
4. Clear output
Show the result in a readable format for support or engineering.
5. Manual review
A human can correct the AI output.

At this stage, the product does not need issue clustering or root-cause reasoning yet. It only needs to reliably turn messy support text into structured data.

7) Features to add step by step

Phase 1: Structured ticket intelligence

Add:

- category detection
- sentiment detection
- severity scoring
- concise ticket summary

This is the first real useful layer. It makes every ticket easier to read and triage.

Phase 2: Knowledge-grounded answers

Add retrieval over a small support knowledge base so the bot can answer common questions with grounding instead of guessing. The uploaded context explicitly includes RAG as a core idea for accurate responses.

Phase 3: Duplicate and pattern detection

Add clustering so similar tickets are grouped together. This is the feature that turns the product from a bot into intelligence. It helps identify repeated issues faster.

Phase 4: Engineering handoff

Add automatic creation of:

- GitHub issue
- Slack/Discord alert
- structured incident summary

For the hackathon, GitHub Issues is the cleanest zero-cost engineering sink.

Phase 5: Impact scoring

Add:

- number of affected users
- urgency score
- customer frustration score
- potential business impact

This gives support and engineering a reason to prioritize one issue over another.

Phase 6: Dashboard

Add a simple view showing:

- top issue clusters
- repeated issue trends
- sentiment trends
- recent escalations
- open incidents

Phase 7: Engineering recommendation layer

Add lightweight AI suggestions such as:

- likely failing module
- probable next debug step
- relevant past issue
- recommended owner team

This should stay advisory, not autonomous.

Phase 8: Calendar integration

Add connectivity to Google Calendar to allow specific reminders and follow-up tasks to be immediately logged from the support ticket interface.

8) Final MVP scope

The final MVP should contain only what is necessary to prove the concept end to end:

- support message input
- AI structuring of the ticket
- sentiment and severity detection
- issue clustering
- knowledge-grounded response
- engineering ticket generation
- simple dashboard

That is enough to tell a strong story:

“Support tickets are not just answered; they are turned into product intelligence.”

9) Recommended zero-cost tool stack

This stack is chosen for no license cost, low setup friction, and local/offline feasibility.

Frontend:

- React
- Vite
- plain CSS or Tailwind if needed

Vite's official docs describe it as a "blazing fast frontend build tool," which makes it a good fit for quick hackathon iteration. ([vitejs](#))

Backend:

- FastAPI
- Python

FastAPI's official docs describe it as a high-performance, easy-to-learn, fast-to-code framework ready for production, and it includes automatic API docs. ([FastAPI](#))

Database:

- SQLite for MVP
- optional PostgreSQL later

Vector store:

- Chroma, run locally

Chroma's docs describe it as open-source AI data infrastructure with embeddings, vector search, metadata filtering, and local/self-hosted use. That makes it a good zero-cost retrieval layer. ([Chroma Docs](#))

LLM:

- Ollama running locally
- a small open model through Ollama

Ollama's site says you can "run entirely offline," your data stays yours, and it is built around open models. That makes it the best zero-cost choice for a hackathon if you want local inference. ([Ollama](#))

Embeddings:

- sentence-transformers locally
- or Ollama-compatible embeddings if available in your environment

Automation:

- GitHub Actions for CI

GitHub Actions docs say it automates workflows directly in the repository, supports jobs/actions, and can run workflows in a customized way. ([GitHub Docs](#))

Engineering handoff:

- GitHub Issues
- Discord webhook as a free alert channel

Deployment for hackathon:

- local demo first
- optional free static hosting for frontend later

10) Success metrics

The product should be measured by:

- structured ticket accuracy
- clustering quality
- average time to triage
- number of duplicate issues detected
- number of engineering-ready summaries produced
- sentiment detection usefulness
- manual correction rate

11) Definition of done

The MVP is done when this works reliably:

Customer message → AI summary → AI classification → AI cluster match → AI issue report → human can review it in one screen.

12) Suggested build order

1. Build the ticket extraction prompt.
2. Store extracted JSON in SQLite.
3. Add the React UI.
4. Add local embeddings and Chroma.
5. Add clustering.
6. Add GitHub issue creation.
7. Add the dashboard.
8. Add knowledge-base retrieval.